

Agentic RAG That Doesn't Hallucinate

Guardrails & Evaluation on the NVIDIA Stack

Wei Cheng (Wayne) Chiu

AI Solutions Engineer

github.com/waynehacking8

The demo that passes, then fails in production

- Your RAG agent answers the 20 questions you tried. Ship it.
- A user asks something **your corpus doesn't cover**.
- A confident, fluent, **wrong** answer comes back.
- In an enterprise setting that's not a bug — it's a **trust incident**.

The hard part of agents isn't getting an answer. It's knowing when to **refuse**.

What "hallucination" means in this talk

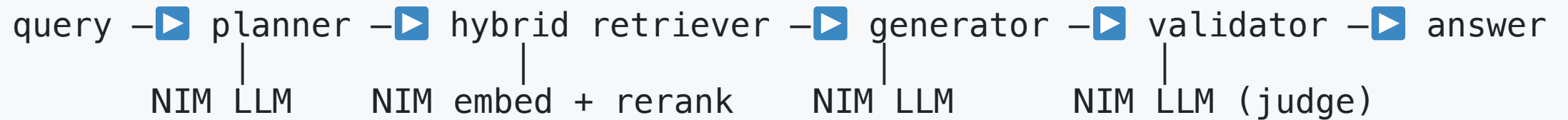
Not vibes — a measurable event:

The agent **asserts** an answer (doesn't abstain) on a question the **retrieval corpus cannot support**.

The nasty case is the **adversarial near-miss**:

- Corpus has **H100** facts.
- I ask about **B200 / H200 / FP4**.
- The model is *tempted* to answer from the H100 facts sitting in context.

The blueprint: plan → retrieve → generate → validate



- **NIM-backed** — one provider switch: hosted `build.nvidia.com` ↔ self-hosted NIM/vLLM.
- `validate()` is the differentiator — every answer is checked for groundedness *before* it's returned.
- OpenTelemetry-traced, so each hop is observable.

Two cheap defenses. No fine-tune.

Defense 1 — guarded generator prompt (*first line*)

"Answer using ONLY the context. Cite [n]. If unsupported, say you don't know."

Defense 2 — `validate()` groundedness gate (*safety net*)

An LLM-as-judge checks the answer against the retrieved context and **blocks** unsupported ones.

Both are prompt-level. Zero training. The question is: **how much do they actually buy?**

You can't claim this without an eval that tries to break it

The eval set is built to be adversarial:

- **20-passage** corpus (NVIDIA inference facts)
- **16 answerable + 10 unanswerable** questions
- unanswerable = out-of-corpus, incl. the **B200/H200/FP4 near-misses**
- two runs per question: **guarded** vs **unguarded** (ablation)

Self-hosted on H100: vLLM **Qwen3-8B** (NIM stand-in) + Ollama embeddings · k=3 ·
temp=0

Headline result

metric	value
retrieval recall@3 (answerable)	100% (16/16)
answer accuracy (answerable)	100% (16/16)
hallucination on unanswerable — guarded	0% (0/10)
hallucination on unanswerable — unguarded (ablation)	40% (4/10)

The same model, same corpus, same questions.

The **only** difference is one instruction in the system prompt.

(N=26 illustrative set — the methodology is the point, not the score.)

The honest part: the judge gate is a *weak* second line

Score `validate()` as a hallucination detector on the **unguarded** run
(where hallucinations actually exist):

	gate blocks	gate passes
hallucinated (should block)	TP = 1	FN = 3
grounded / abstained (should pass)	FP = 1	TN = 21

precision 50% · recall 25% · F1 0.33 → residual hallucination **40% → 30%**

A single-pass LLM judge is **not** enough on its own. It's a net, not a wall.

(The judge shares the generator's model — self-grading bias is part of why recall is low.)

The agentic tax: per-hop latency budget

plan	retrieve	generate	validate	total
0.24s	0.165s	0.30s	0.16s	0.87s

- `plan` + `validate` = **two extra LLM calls** wrapping every answer.
- `retrieve` is cheap (embed + in-memory fuse).
- The reliability you saw isn't free — it's **~0.40s of extra LLM calls** per query.

Reproduce it yourself (≈5 min)

```
git clone https://github.com/waynehacking8/nim-agent-blueprint
cd nim-agent-blueprint

# point at any OpenAI-compatible endpoint (hosted NIM, or local vLLM/Ollama)
export NIM_MODE=selfhost NIM_LLM_URL=http://localhost:8000/v1
python eval/run_eval.py          # regenerates eval/report.md
```

Flip `guarded=False` in the harness and watch the 0% become 40%.

Full walkthrough: `talk/walkthrough.md`

Takeaways

1. In enterprise RAG, **refusing** is a feature. Measure it.
2. Your eval must include questions the corpus **can't** answer — incl. near-misses.
3. A **guarded prompt** is the cheapest, biggest win. (40% → 0% here.)
4. An **LLM-judge gate** is a backstop, not a guarantee. Know its recall.
5. Reliability has a **latency cost** — quote it at design time.

Repo: github.com/waynehacking8/nim-agent-blueprint

Me: waynehacking8.github.io

Q&A

Where to go deeper in the repo:

- `app/agent.py` — the `plan → retrieve → generate → validate` loop
- `eval/run_eval.py` — how every number on these slides is computed
- `eval/report.md` — the full per-question breakdown
- `docs/design-decisions.md` — why guarded-by-default, why a judge gate
- `docs/roadmap.md` — hardening the gate, hosted-NIM run, multi-hop

Thank you.